

Research notes and experiment planning on LaCam_T

Abstract—Multi-agent path finding with turn actions (MAPF_T) is an important extension of classical MAPF for mobile robots with orientation and turn-delay constraints. Our previous solver, PIBT_T^{*}, can handle many large-scale MAPF_T instances efficiently, but it does not provide completeness or optimality guarantees. In this paper, we take a step toward eventually complete and optimal planning in MAPF_T by developing LaCAM_T, an orientation-aware extension of the LaCAM framework. The proposed method adopts a two-level search architecture, where the high level searches over joint configurations and the low level generates feasible successor configurations under turn-constrained motion. A key challenge in this setting is that the introduction of orientations significantly enlarges the search space and slows down solution refinement after the first feasible solution is found. To address this issue, we focus on the design of the high-level node extraction strategy, aiming to improve convergence speed without introducing excessive selection overhead. In particular, we incorporate a lightweight multi-armed-bandit-based mechanism into the high-level search and study its role in guiding refinement. The resulting framework establishes the basis of LaCAM_T and provides a foundation for investigating scalable, eventually complete, and eventually optimal planning for large-scale MAPF_T problems.

Note to Practitioners—In practical multi-robot systems such as automated warehouses, flexible manufacturing systems, and robotized transport platforms, mobile robots often need to rotate before moving in the desired direction. If these turn actions are ignored during planning, the resulting paths may appear feasible at the route level but can lead to overly optimistic schedules and inefficient execution in practice. Our previous solver, PIBT_T^{*}, addressed this issue from the perspective of scalable real-time replanning and was able to handle many large-scale turn-constrained instances efficiently. However, it does not provide completeness or optimality guarantees, which limits its applicability in scenarios where solution quality and theoretical assurance are both important.

This paper takes a step toward that direction by developing LaCAM_T, a two-level turn-aware search framework derived from LaCAM. From a practitioner’s perspective, an important message of this work is that, once turn actions are explicitly modeled, the challenge is no longer only to generate feasible local motions, but also to identify which global search states deserve further refinement. In other words, the convergence speed of the planner depends heavily on the high-level node extraction strategy. Our implementation therefore studies a lightweight bandit-guided mechanism for selecting high-level nodes, aiming to improve refinement efficiency without introducing excessive overhead. The resulting framework is relevant to applications that require both realistic motion modeling and progressively improving solution quality under limited planning time. More broadly, this work suggests that, for large-scale turn-constrained multi-robot planning, careful high-level search control can be as important as low-level conflict resolution.

Index Terms—Multi-agent path finding, collision avoidance, motion and path planning, turn actions

I. SEVERAL OPTIONS TO IMPROVE LACAM

Table I summarizes the relationship between the engineering techniques introduced in Okumura’s enhanced LaCAM^{*} framework and their potential extensions to LaCAM_T. A clear observation is that most of Okumura’s improvements do not explicitly focus on fine-grained high-level node valuation. Instead, they mainly improve search performance through lightweight escape strategies at the high level, better successor generation at the low level, and the incorporation of alternative solutions during the anytime refinement process. In particular, the non-deterministic node extraction mechanism improves robustness by helping the search escape stuck regions, while SUO and Monte-Carlo configuration generation improve the quality of the generated successors. These techniques are highly effective, but they do not directly address the question of how to distinguish more promising high-level nodes during refinement.

This observation is particularly important in MAPF_T. Compared with position-only MAPF, the high-level search space in LaCAM_T is more heterogeneous because node quality is influenced not only by agent locations, but also by orientations, turn requirements, and the resulting downstream interaction patterns. As a result, a coarse DFS-like extraction rule or a purely random escape strategy may be insufficient to fully exploit the structure of the search space during the optimization phase. This suggests that high-level node selection itself may constitute an additional opportunity for improving refinement efficiency in LaCAM_T, provided that the selector remains lightweight enough to preserve the scalability advantage of the LaCAM family.

Based on this comparison, our position is not to replace Okumura-style enhancements, but rather to complement them. Specifically, we view high-level node selection as a missing layer between coarse search control and low-level successor generation. The objective is therefore to investigate whether a lightweight, feature-informed node selection mechanism can improve refinement efficiency beyond coarse stochastic extraction, while remaining compatible with other engineering enhancements such as Monte-Carlo successor generation or turn-aware low-level guidance.

II. CURRENT IMPLEMENTATION: LACAM_T WITH MAB FOR NODE EXTRACTION

III. CURRENT IMPLEMENTATION

This section summarizes the current implementation of LaCAM_T with MAB-based high-level node extraction. The implementation is built upon LaCAM2 and extends it to an orientation-aware setting, while preserving the overall

This research was supported by [Funding Agency] under Grant [Grant Number]. (Corresponding author: [Corresponding Author Name].)

[Author A] and [Author B] are with the [Department Name], [University Name], [City], [Country] (e-mail: [authorA@university.edu]; [authorB@university.edu]).

TABLE I: Comparison between the enhancement techniques in Engineering LaCAM* and potential extensions for LaCAM_T

Technique	Main role in Okumura's LaCAM*	Where it acts	Can it be directly reused in LaCAM _T ?	Comments for LaCAM _T + MAB
Non-deterministic node extraction	Escape search-stuck situations after the first feasible solution is found; mainly implemented by restart/random extraction with a small probability	High-level node extraction	Yes, with minimal modification	This is the most direct baseline for comparison with MAB. It is lightweight and preserves scalability, but it does not explicitly distinguish promising high-level nodes. It can be viewed as a low-cost escape mechanism rather than a value-aware selector.
Space Utilization Optimization (SUO)	Improve the quality of the candidate ordering in PIBT by biasing agents toward spatially dispersed reference paths	Low-level configuration generation	Partially	The general idea is reusable, but the path representation must be adapted to orientation-aware states. In MAPF _T , this idea may become even more valuable because path smoothness and turning structure matter in addition to spatial dispersion.
Monte-Carlo configuration generation	Sample multiple successor configurations from the same node and choose the best one according to a lightweight evaluation rule	Low-level successor generation	Yes	This is highly compatible with LaCAM _T . It does not compete with MAB; instead, it complements MAB by improving successor quality after a high-level node has been selected. This may be a strong second line of improvement if high-level node selection alone is insufficient.
Dynamic incorporation of alternative solutions	Inject improved solutions found by external refiners into the current search graph	High-level graph update / anytime refinement	Yes, in principle	This is more complex to implement but conceptually attractive. For LaCAM _T , one can consider injecting alternative orientation-aware solution segments obtained by local repair or partial replanning. This is orthogonal to MAB and can be added later.
Recursive call of LaCAM*	Use LaCAM* itself as a refiner on a subproblem starting from an intermediate configuration	External refinement + solution injection	Partially	The idea is reusable, but the cost will be higher in MAPF _T due to the enlarged state space. It is probably not the first direction to try. It becomes more meaningful after the core LaCAM _T framework is already stable.

high-level/low-level decomposition of the original framework. Compared with the position-only version, the current implementation explicitly models agent orientations, turn actions, and turn-induced waiting behaviors, and introduces a two-stage high-level search strategy: a depth-first feasibility phase followed by a UCB-based optimization phase.

A. Orientation-aware state space and action model

In the current implementation, each agent state is represented by a position–orientation pair

$$s = (v, o),$$

where v is a traversable vertex in the map graph and o is one of the four cardinal orientations. Accordingly, a configuration is defined as the collection of all agents' states, and a solution is a sequence of such configurations over time. This differs from the original LaCAM2 implementation, where a state only stores the vertex location.

The action space is also extended to reflect turn-constrained motion. At each timestep, an agent may execute one of four

actions: waiting in place, turning left in place, turning right in place, or moving one step forward along its current heading. A forward motion is only allowed when the next vertex lies exactly in the current facing direction. Therefore, unlike in classical MAPF, a desired translational move may require one or more preparatory turning actions before it can be executed.

B. Orientation-aware heuristic table

To support efficient search in the enlarged state space, the current implementation constructs an orientation-aware distance table for each agent. Instead of storing one distance value per vertex, the table stores one value for each vertex–orientation pair. These values are computed by reverse breadth-first search from the goal location. The reverse expansion includes both rotational predecessors at the same vertex and translational predecessors consistent with the forward-motion model. Since the goal test is defined on target positions only, all four orientations at the goal vertex are initialized with zero distance. This implementation adopts eager preprocessing, so

that each heuristic lookup during search can be performed in constant time.

C. High-level node representation

A high-level node in the current implementation represents a global orientation-aware configuration together with the data required for successor generation and cost propagation. In addition to the current configuration C , each node stores the path cost g , the heuristic value h , the evaluation value $f = g + h$, a pointer to its parent, a list of neighboring high-level nodes for rewrite propagation, an agent priority vector, the corresponding priority order, and a low-level search tree that records partially assigned successor states. This design follows the original LaCAM philosophy of combining a configuration-level search space with lazy successor generation, while extending the node contents to support turn-aware planning.

D. Two-stage high-level search with MAB-based node extraction

The current implementation uses a two-stage high-level search strategy. Before any feasible solution is found, the OPEN list is managed as a double-ended queue and expanded in a depth-first manner by always selecting the node at the back. The purpose of this stage is to quickly identify an initial feasible solution.

After the first feasible solution is obtained, the planner switches to an optimization stage. At this stage, the OPEN list is partitioned into three consecutive regions, referred to as Phase 0, Phase 1, and Phase 2. A multi-armed bandit mechanism based on the upper confidence bound (UCB) rule is then used to choose one region, after which one node is sampled uniformly from the selected region for expansion. Let $\bar{r}_i$ denote the average reward of region i , n_i the number of times region i has been selected, and n the total number of selections. The UCB score is computed as

$$\text{UCB}(i) = \bar{r}_i + c \sqrt{\frac{\ln(n+1)}{n_i}},$$

where c is the exploration coefficient.

The reward design in the current implementation is outcome-based. Larger rewards are assigned when the selected node leads to a better incumbent solution or to a more promising successor, while smaller rewards are assigned to ordinary node generation or rewrite events. If successor generation fails, the reward is set to zero. The current implementation therefore uses MAB not as a full node-ranking mechanism, but as a coarse region selector intended to improve refinement efficiency without introducing expensive global sorting over the entire OPEN list.

E. Low-level search tree and successor generation

Successor configurations are generated lazily through a low-level search tree. Each low-level node records a partial assignment of successor states for a subset of agents. During expansion, the next unconstrained agent is chosen according to the priority order stored in the current high-level node, and all

legal one-step successor states are enumerated, including wait, left turn, right turn, and forward motion. These candidate states are randomly shuffled before being inserted into the low-level queue, thereby preserving a degree of diversity in successor generation.

Given a partially constrained low-level node, the procedure GETNEWCONFIG first applies all fixed assignments and performs early checks for obvious vertex conflicts and swap conflicts. The remaining agents are then processed by a PIBT-based recursive procedure. If a complete assignment is obtained, the resulting configuration is validated against vertex conflicts, swap conflicts, and turn-aware physical feasibility. Only valid configurations are accepted as successors.

F. PIBT-based recursive assignment under turn constraints

The core low-level generator is an orientation-aware adaptation of PIBT. For each agent, the recursive routine constructs a candidate list containing all neighboring vertices together with the current vertex. These candidates are ordered greedily according to turning cost and goal-oriented heuristic distance. The routine then tries to reserve one candidate at a time while recursively relocating blocking agents when necessary.

A key difference from position-only PIBT is that candidate selection and action execution are separated. Even when a target vertex is chosen, the actual one-step action may still be an in-place turn if the agent is not yet aligned with the target direction. In such cases, the implementation records the desired future target in a reserved-node structure. During the next planning step, this reserved target is prioritized again, thereby allowing the agent to continue the intended motion after completing the required turn. This reserved-node mechanism is important for preserving directional consistency across consecutive steps.

G. Additional conflict-handling mechanisms

To improve robustness in the turn-constrained setting, the implementation includes several additional mechanisms beyond the basic PIBT recursion.

First, a swap-handling mechanism is used to address narrow-passage exchanges. It checks whether a swap is both necessary and feasible, and if so, triggers an explicit swap-oriented resolution procedure. Second, a cycle-handling mechanism is used when the recursive request chain forms a cycle. If some agents in the cycle are not yet aligned with their intended directions, they first rotate in place while the others wait; once all are aligned, the cycle is executed simultaneously. Third, a k -push escape trigger is used to reduce repeated local pushing. It records how many times one agent has pushed another and randomizes the candidate order if the same pushing pattern occurs too often. Together, these mechanisms are designed to mitigate deadlocks, cyclic waiting, and repeated local oscillation caused by turn actions.

H. Rewrite-based cost propagation

When a newly generated configuration has already been explored, the implementation does not discard it immediately.

Instead, it invokes a rewrite procedure to determine whether the known high-level node can be reached with a lower path cost through the current parent. If so, the parent relation and the corresponding g -value are updated, and the improvement is propagated to neighboring nodes through a Dijkstra-style traversal over the explored high-level graph. Nodes whose updated f -values become sufficiently promising may be reinserted into OPEN. This mechanism allows the current implementation to preserve the eventual-improvement character of LaCAM-style search while exploiting newly found lower-cost paths.

I. Current implementation status

Overall, the current implementation already realizes the main building blocks of LaCAM_T: orientation-aware state representation, turn-aware heuristic computation, lazy low-level successor generation, PIBT-based recursive conflict resolution, rewrite-based cost propagation, and a two-stage high-level search with MAB-guided node extraction. The current MAB component is intentionally lightweight: it only selects among three coarse regions of the OPEN list, while the final node within the chosen region is still sampled randomly. This design keeps the per-selection overhead low, but it also suggests that further improvement may depend on whether more informative yet still inexpensive high-level node selection rules can be developed in future work.

IV. EXPERIMENTAL GUIDELINE FOR VALIDATING HIGH-LEVEL NODE SELECTION

This subsection provides an experimental guideline for validating the design of the high-level node selection mechanism in LaCAM_T. The goal is not to immediately introduce a complicated selector, but to identify, in a controlled and incremental manner, whether the observed performance bottleneck mainly comes from: (i) overly coarse arm construction, (ii) excessive randomness within each arm, or (iii) the lack of turn-aware structural information in the current selection process.

A. Overall validation objective

The experiments are designed to answer the following question:

Can a lightweight, feature-informed high-level node selection mechanism improve the refinement efficiency of LaCAM_T without sacrificing scalability?

To answer this question, the validation is organized into three stages. The first stage examines whether the current degradation mainly comes from random node extraction within a selected arm. The second stage evaluates whether the current arm definition is too coarse. The third stage investigates whether cheap MAPF_T-specific features can further improve the selection quality.

B. Common experimental setting

All compared variants should use the same low-level configuration generator, the same rewrite procedure, the same heuristic table, and the same time limit. To isolate the effect of

the high-level node selection strategy, all other implementation details should remain unchanged. The compared methods should be evaluated on the same benchmark instances and random seeds.

For each run, the following quantities should be recorded:

- (1) the time to the first feasible solution;
- (2) the incumbent objective value as a function of runtime;
- (3) the objective value at several fixed runtime checkpoints (e.g., 1 s, 3 s, 5 s, 10 s, and 30 s);
- (4) the number of high-level node expansions;
- (5) the number of successor generations;
- (6) the average time spent in node selection;
- (7) the average time spent in successor generation;
- (8) the percentage of total runtime consumed by the selection module.

The main evaluation criterion should be *improvement per second*, rather than only improvement per expansion. This is important because a more informed selector may improve decision quality while also introducing additional overhead.

C. Stage I: validating the effect of intra-arm randomization

The first stage keeps the current MAB framework unchanged and only modifies the node extraction rule within a selected arm. The purpose is to determine whether the current performance loss mainly comes from pure random extraction inside the arm.

Three variants should be compared:

- (1) **Random-in-arm**: the current baseline, where MAB selects an arm and a node is sampled uniformly at random from that arm;
- (2) **Best- f -in-sample**: MAB first selects an arm, then k nodes are sampled uniformly from that arm, and the node with the smallest f -value is chosen;
- (3) **Best- h -in-sample**: MAB first selects an arm, then k nodes are sampled uniformly from that arm, and the node with the smallest h -value is chosen.

The sample size k should be chosen from a small set such as

$$k \in \{4, 8, 16\}.$$

This keeps the additional overhead small while allowing the selector to exploit partial information inside the arm.

a) *Expected interpretation.*: If either **Best- f -in-sample** or **Best- h -in-sample** consistently outperforms **Random-in-arm**, then the main weakness of the current design is likely not the MAB framework itself, but rather the loss of useful signal caused by uniform random extraction within each arm.

D. Stage II: validating the effect of arm granularity

The second stage examines whether the current arm definition is too coarse. To isolate this factor, the intra-arm rule should be fixed, for example by using the best-performing rule identified in Stage I.

The following arm granularities should be compared:

$$B \in \{3, 6, 12\},$$

where B is the number of arms. Under this setting, the current OPEN list can still be partitioned according to its positional order, but with a finer discretization.

a) *Expected interpretation.*: If increasing the number of arms improves the refinement performance, then the current degradation on large maps is likely caused by insufficient arm granularity. In other words, the current three-arm partition is too coarse to preserve meaningful differences among high-level nodes in large-scale instances.

E. Stage III: validating the effect of semantic arm construction

If Stage II suggests that the arm construction is too coarse, the next step is to replace positional partitioning with low-cost semantic partitioning. The purpose is to determine whether arms should be defined by node properties rather than by their position in the OPEN list.

A first set of semantic buckets can be defined using only cheap search attributes:

- (1) **f -based buckets**: low- f , medium- f , and high- f nodes;
- (2) **h -based buckets**: low- h , medium- h , and high- h nodes;
- (3) **rewrite-status buckets**: nodes recently involved in rewrite propagation versus others.

For each semantic partitioning rule, the same MAB mechanism should be retained, and only the arm construction should be changed.

a) *Expected interpretation.*: If semantic buckets outperform positional buckets, then the main issue is not merely the number of arms, but rather the lack of meaningful semantics in the current arm definition. This would support the view that MAB should operate on informative node groups rather than on arbitrary positional slices of the OPEN list.

1) *Stage IV: introducing MAPF_T-specific cheap features*: After identifying a reasonable semantic bucket structure, the next stage is to examine whether MAPF_T-specific structural information can further improve the selector. The emphasis here should remain on *cheap* features that can be computed or maintained incrementally.

Candidate features include:

- (1) the proportion of agents whose orientations are not aligned with their locally desired directions;
- (2) the number of agents that still require additional turn actions before forward progress;
- (3) the activity level of reserved-node usage;
- (4) a recent push/cycle signal, such as whether the node expansion history involves frequent push propagation or cycle handling;
- (5) a lightweight local congestion indicator derived from the current configuration.

At this stage, two design choices can be evaluated:

- (1) **Turn-aware semantic buckets**: use one or two turn-aware features to define the arms;
- (2) **Turn-aware intra-arm scoring**: keep the arm definition fixed, but augment the intra-arm sampling score, e.g.,

$$\text{score}(H) = -f(H) - \lambda \cdot \phi(H),$$

where $\phi(H)$ is a cheap turn-aware penalty and λ is a scalar coefficient.

a) *Expected interpretation.*: If these turn-aware features improve performance with only marginal overhead, then the high-level selection problem in LaCAM_T indeed benefits from information that is absent in position-only LaCAM-style planning. This would provide a strong justification for a turn-aware node selection mechanism as a methodologically meaningful contribution.

F. Suggested comparison protocol

To keep the study controlled and interpretable, the following comparison order is recommended:

- (1) current MAB baseline with random-in-arm extraction;
- (2) MAB with local sampled scoring inside each arm;
- (3) MAB with finer positional partitioning;
- (4) MAB with semantic buckets based on f or h ;
- (5) MAB with MAPF_T-specific cheap features.

Each new variant should only modify one component relative to the previous stage. This one-factor-at-a-time protocol is important for identifying the actual source of improvement.

G. Result reporting recommendation

The results should be reported from both effectiveness and efficiency perspectives.

a) *Effectiveness.*: For each method, plot the incumbent objective value against runtime. In addition, report the average objective value at fixed checkpoints and the average percentage improvement over the baseline at the time limit.

b) *Efficiency.*: For each method, report:

- (1) average node-selection time per expansion;
- (2) average successor-generation time per expansion;
- (3) number of expansions per second;
- (4) fraction of total runtime spent on the selector.

c) *Decision criterion.*: A variant should be regarded as successful only if it improves the incumbent quality under the same wall-clock budget without introducing a prohibitive overhead. Therefore, the final judgment should be based on the trade-off between refinement quality and runtime cost, rather than on decision quality alone.

H. Practical stopping rule for method development

The development process can follow the following stopping rule:

- (1) If Stage I already yields clear gains, retain MAB and improve only the intra-arm policy;
- (2) If Stage II yields gains but Stage I does not, focus on better arm granularity rather than better local scoring;
- (3) If Stage III yields gains over Stage II, prioritize semantic arm construction;
- (4) If Stage IV yields only marginal gains with noticeable overhead, stop at Stage III and keep the method lightweight.

In this way, the experimental study can identify not only whether a more informative high-level selector is useful, but also how far such refinement should go before it begins to undermine the scalability advantage of LaCAM_T.

V. GLOBAL LOWER BOUNDS FOR ANYTIME OPTIMALITY CERTIFICATION

A. Motivation

LaCAM_T is an anytime, eventually optimal planner: after the first feasible solution is found, the optimization stage keeps refining the incumbent, and optimality is only established once the search space has been exhausted. In the turn-aware setting this exhaustion can be prohibitively slow, because orientations enlarge the high-level search space and the refinement phase may spend a long time proving that no better solution exists. Consequently, when the planner is interrupted under a time budget, it returns a solution whose *quality is unknown*: the incumbent may already be optimal, or still far from it, and the planner cannot tell the two cases apart.

A valid *global lower bound* on the optimal cost addresses this limitation directly. Let OPT denote the optimal objective value and let UB_t denote the incumbent cost at time t (which is an upper bound, $UB_t \geq \text{OPT}$). If we can also maintain a lower bound $LB_t \leq \text{OPT}$, then at any time we obtain the two-sided certificate

$$LB_t \leq \text{OPT} \leq UB_t, \quad \text{gap}_t = \frac{UB_t - LB_t}{UB_t}.$$

Such a bound enables three capabilities that the original scheme lacks. First, it turns any interrupted run into a *bounded-gap* solution: the planner may stop as soon as $\text{gap}_t \leq \epsilon$ and report a certified ϵ -optimal solution. Second, it enables *early optimality certification*: whenever $LB_t = UB_t$ the incumbent is provably optimal and the search can terminate without exhausting the OPEN list. Third, it provides a principled progress signal for monitoring convergence and for guiding refinement. The remainder of this section develops three lower bounds of increasing looseness but decreasing cost, and proves that each is valid.

B. Preliminaries and notation

At planning time t , let $H_t = (V_t, E_t)$ denote the *explored* high-level graph, where V_t is the set of discovered configurations and E_t is the set of transitions that have already been generated by the low-level procedure. Let $S \in V_t$ be the start configuration and let $\mathcal{G}$ be the set of goal configurations (those in which every agent occupies its goal vertex). For a configuration X we write $X_i = (v_i, o_i)$ for the state of agent i and $D_i(\cdot)$ for the orientation-aware single-agent distance-to-goal table of Section III-B. Transition costs are denoted $c(X, Y) \geq 0$; for the makespan objective every one-step transition has unit cost.

We use the standard search quantities of Section III-C. For a discovered configuration $X \in V_t$, $g_t(X)$ is the cost of the shortest *known* path from S to X inside H_t , maintained exactly by the rewrite propagation of Section III-H; $h(X)$ is the heuristic value; and $f_t(X) = g_t(X) + h(X)$. The incumbent cost is $UB_t = \min\{g_t(Z) : Z \in \mathcal{G} \cap V_t\}$ (with $UB_t = +\infty$ before the first solution). We rely on two standard properties.

Lemma 1 (Admissibility of h). *For every configuration X , $h(X)$ is a lower bound on the optimal cost of completing the plan from X to a goal configuration.*

Proof. The heuristic ignores inter-agent interaction: $D_i(X_i)$ is the exact cost of the turn-aware shortest path that agent i alone would take from X_i to its goal, which no collision-free joint plan can undercut for that agent. For the makespan objective $h(X) = \max_i D_i(X_i)$, and since the makespan of any completion is at least the arrival time of each individual agent, it is at least $\max_i D_i(X_i)$. For an additive objective $h(X) = \sum_i D_i(X_i)$, and since the total cost is the sum of the individual costs, it is at least $\sum_i D_i(X_i)$. Removing the collision constraints can only decrease the optimal completion cost, so $h(X)$ never overestimates it. $\square$

Lemma 2 (Exactness of g_t on H_t). *After rewrite propagation has settled, $g_t(X)$ equals the cost of a shortest path from S to X using only edges of E_t . In particular, for any path from S to X all of whose edges lie in E_t , its cost is at least $g_t(X)$.*

Proof. Newly created configurations receive g equal to the parent's g plus the generating edge cost, which is exact because a brand-new configuration has a unique known incoming edge. Whenever a transition is (re)generated into an already-known configuration, the rewrite procedure performs a Dijkstra-style relaxation over E_t and re-propagates every improvement, so on settling g_t is a fixed point of the Bellman conditions on H_t , i.e., the shortest-path function on H_t . The second claim is immediate since the given path is one particular S - X path in H_t . $\square$

Finally, let $\mathcal{F}_t \subseteq V_t$ be the *frontier*: the set of discovered configurations whose successors have not yet been fully generated, i.e., whose low-level search tree of Section III-E is non-empty. Equivalently, $X \in \mathcal{F}_t$ iff there exists at least one legal successor of X not yet in E_t .

C. A frontier-based global lower bound

The natural candidate $\min_{X \in \text{OPEN}} f_t(X)$ is *not* a valid global lower bound under lazy successor generation, because $g_t(X)$ reflects only the currently known graph H_t and may exceed the true shortest-path cost to X in the complete configuration graph. The correct construction instead reasons about the *first not-yet-generated edge* along any complete path. For a frontier configuration X , let $U_t(X) = \text{Succ}(X) \setminus \text{Gen}_t(X)$ denote its ungenerated successors and define the one-step frontier extension

$$\ell_t(X) = \min_{Y \in U_t(X)} [c(X, Y) + h(Y)].$$

Theorem 1 (Frontier lower bound). *Let*

$$LB_t^{\text{fr}} = \min\left\{UB_t, \min_{X \in \mathcal{F}_t} [g_t(X) + \ell_t(X)]\right\}.$$

Then $LB_t^{\text{fr}} \leq \text{OPT}$.

Proof. Let $\pi : S \rightarrow \dots \rightarrow Z^*$ be an optimal complete path, with $Z^* \in \mathcal{G}$ and $\text{cost}(\pi) = \text{OPT}$. We distinguish two cases.

Case 1: every edge of π lies in E_t . Then π is a known S - Z^* path, so by Lemma 2 $g_t(Z^*) \leq \text{cost}(\pi)$, and since $Z^* \in \mathcal{G} \cap V_t$ we have $UB_t \leq g_t(Z^*) \leq \text{OPT}$. Hence $LB_t^{\text{fr}} \leq UB_t \leq \text{OPT}$.

Case 2: some edge of π is not in E_t . Let $X \rightarrow Y$ be the first such edge along π . All edges of the prefix $S \rightarrow \dots \rightarrow X$ are then generated, so this prefix is a path in H_t and, by Lemma 2,

$$g_t(X) \leq \text{cost}(S \rightarrow X \text{ along } \pi).$$

Since $Y \in U_t(X)$ and, by Lemma 1, $h(Y) \leq \text{cost}(Y \rightarrow Z^* \text{ along } \pi)$,

$$\ell_t(X) \leq c(X, Y) + h(Y) \leq \text{cost}(X \rightarrow Z^* \text{ along } \pi).$$

Adding the two inequalities gives $g_t(X) + \ell_t(X) \leq \text{cost}(\pi) = \text{OPT}$. Because X has the ungenerated successor Y , we have $X \in \mathcal{F}_t$, hence $\min_{X' \in \mathcal{F}_t} [g_t(X') + \ell_t(X')] \leq g_t(X) + \ell_t(X) \leq \text{OPT}$, and therefore $\text{LB}_t^{\text{fr}} \leq \text{OPT}$. $\square$

D. A cheap global lower bound

The bound of Theorem 1 is tight but requires evaluating $\ell_t(X)$, which enumerates the ungenerated successors $U_t(X)$ of each frontier configuration. Doing so at every step would negate the lazy-successor-generation advantage that underlies the scalability of LaCAM_T. We therefore relax the one-step extension $\ell_t(X)$ to the heuristic $h(X)$ itself, which is already stored on every node and needs no successor enumeration.

Theorem 2 (Cheap lower bound). *Let*

$$\text{LB}_t^{\text{cheap}} = \min \left\{ \text{UB}_t, \min_{X \in \mathcal{F}_t} [g_t(X) + h(X)] \right\} \quad (1)$$

$$= \min \left\{ \text{UB}_t, \min_{X \in \mathcal{F}_t} f_t(X) \right\}. \quad (2)$$

Then $\text{LB}_t^{\text{cheap}} \leq \text{OPT}$. Moreover, if h is consistent then $\text{LB}_t^{\text{cheap}} \leq \text{LB}_t^{\text{fr}}$, i.e., the cheap bound is valid but never tighter than the frontier bound.

Proof. Validity follows the proof of Theorem 1 verbatim in Case 1. In Case 2, with $X \rightarrow Y$ the first ungenerated edge along an optimal path π , Lemma 1 gives directly $h(X) \leq \text{cost}(X \rightarrow Z^* \text{ along } \pi)$, so

$$g_t(X) + h(X) \leq \text{cost}(\pi) = \text{OPT},$$

and $X \in \mathcal{F}_t$ as before; hence the minimum over $\mathcal{F}_t$ is at most OPT. For the comparison, consistency of h gives $h(X) \leq c(X, Y) + h(Y)$ for every successor Y , so $h(X) \leq \ell_t(X)$ term by term, whence $\text{LB}_t^{\text{cheap}} \leq \text{LB}_t^{\text{fr}}$. $\square$

The cheap bound is the version implemented in LaCAM_T. It requires only the f -values of the unfinished frontier nodes, which are already maintained by the search, and is therefore evaluated with negligible overhead. When the search terminates by exhausting the frontier, $\mathcal{F}_t = \emptyset$ and $\text{LB}_t^{\text{cheap}} = \text{UB}_t$, recovering the standard exhaustion-based optimality certificate as a special case.

Corollary 1 (Early optimality by bound). *If at some time t we have $\text{LB}_t^{\text{cheap}} \geq \text{UB}_t$, then the incumbent is optimal, i.e., $\text{UB}_t = \text{OPT}$, even though the frontier $\mathcal{F}_t$ may be non-empty.*

Proof. By Theorem 2, $\text{LB}_t^{\text{cheap}} \leq \text{OPT}$, and the incumbent is a feasible solution so $\text{OPT} \leq \text{UB}_t$. The hypothesis $\text{LB}_t^{\text{cheap}} \geq \text{UB}_t$ then forces $\text{UB}_t \leq \text{OPT} \leq \text{UB}_t$, i.e., equality. $\square$

E. A static two-agent floor bound

The two search-dependent bounds above are anchored at the frontier and may remain loose while the frontier is dominated by early, low- g configurations. We therefore add a *static* bound that is independent of the search state and can be computed once, before search, from the instance alone. We state it for the makespan objective. Let $d_i = D_i(S_i)$ be the single-agent turn-aware distance of agent i from its start to its goal, and for a pair of agents (i, j) let $M_2(i, j)$ be the optimal makespan of the two-agent subproblem that contains only agents i and j and ignores all other agents.

Theorem 3 (Pairwise floor bound). *For the makespan objective,*

$$\text{LB}^{\text{pair}} = \max \left\{ \max_i d_i, \max_{i < j} M_2(i, j) \right\} \leq \text{OPT}.$$

Proof. Let Π be an optimal joint plan with makespan OPT. Fix any agent i ; its trajectory in Π is a turn-feasible walk from S_i to its goal, so its length is at least the single-agent optimum d_i , and since the makespan is the maximum arrival time over all agents, $\text{OPT} \geq d_i$. Taking the maximum over i yields $\text{OPT} \geq \max_i d_i$.

Now fix a pair (i, j) and restrict Π to agents i and j . Because Π is collision-free, the restricted pair of trajectories is free of vertex and swap conflicts *between i and j* , respects the turn-aware motion model, and brings both agents from their starts to their goals by time OPT. It is therefore a feasible plan for the two-agent subproblem, whose optimum $M_2(i, j)$ can only be smaller: $M_2(i, j) \leq \text{OPT}$. Taking the maximum over pairs gives $\max_{i < j} M_2(i, j) \leq \text{OPT}$. Combining the two parts, $\text{LB}^{\text{pair}} \leq \text{OPT}$. $\square$

In the implementation, $M_2(i, j)$ is computed by an exact A^* search over the joint two-agent state space $((v_i, o_i), (v_j, o_j))$, using the same turn-aware action model and unit transition cost as the main search and the admissible heuristic $\max\{D_i, D_j\}$. Since the makespan lower bound $\max_i d_i$ already coincides with the heuristic at the root, only genuinely interacting pairs can raise the bound; pairs whose start-goal regions are spatially disjoint cannot, and are pruned cheaply. The computation is capped by a time budget, and evaluating only a subset of pairs is still a valid (merely looser) bound, since the maximum over a subset of pairs never exceeds the maximum over all pairs.

F. Combined bound and anytime termination

Because the maximum of valid lower bounds is again a valid lower bound, the individual bounds are combined by

$$\text{LB}_t = \max \{ \text{LB}_t^{\text{cheap}}, \text{LB}^{\text{pair}} \} \leq \text{OPT},$$

where LB_t^{fr} may be substituted for, or maximized with, $\text{LB}_t^{\text{cheap}}$ when the tighter frontier evaluation is affordable. Since each valid bound observed over time is itself a lower bound on the fixed value OPT, their running maximum is monotone non-decreasing and remains valid, yielding a tightening lower-bound curve. Together with the incumbent UB_t , this produces the anytime certificate $\text{LB}_t \leq \text{OPT} \leq \text{UB}_t$:

the search may stop with a certified ϵ -optimal solution once $\text{gap}_t \leq \epsilon$, and may declare exact optimality as soon as $\text{LB}_t = \text{UB}_t$ (Corollary 1), in general strictly before the search space is exhausted.